ECE3574 - FL26
Milestone 3 : Qt AI Workspace

Build an event-driven desktop interface for selected AI Workspace features.

Released: October 3, 2026 | Due: October 24, 2026

Milestone 3 Overview

In Milestones 1 and 2, you developed the core AI Workspace functionality for storing documents and processing them to build a corpus, search, and construct context. In M3, you will place a Qt desktop interface around selected parts of that existing functionality. The M1/M2 classes remain the backend; M3 adds the graphical, event-driven layer that lets a user work with them interactively.

The goal is not to redesign the processing system or create a GUI for every feature from earlier milestones. You will build one focused application workflow: add and inspect documents, build the corpus, search it, inspect search results and their source documents, inspect term statistics, and build context.

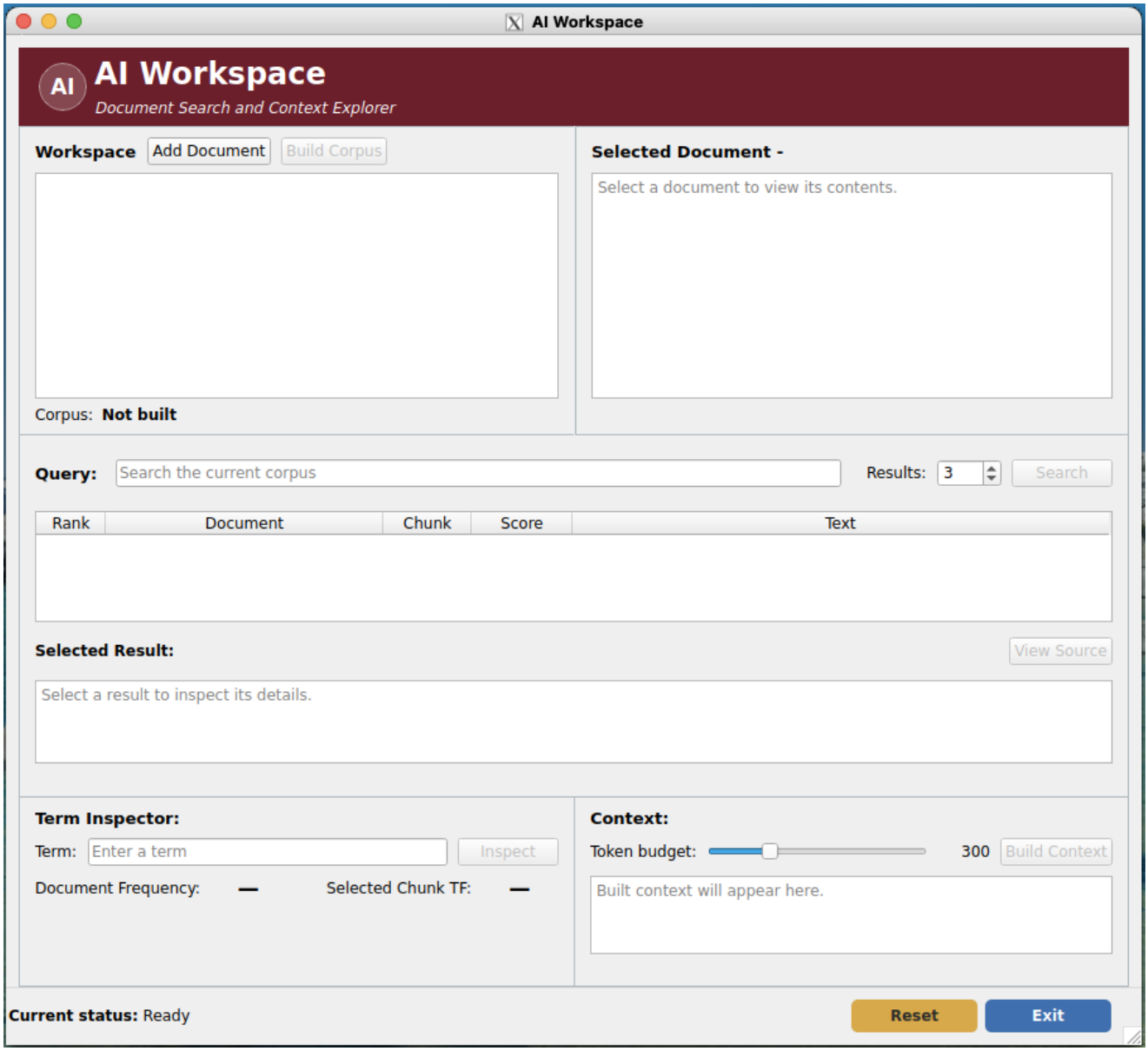


Figure 1. M3 reference application. Follow the overall structure and organization; exact colors are not required.

M3 also gives you practical experience presenting software in a clear and professional way. You are not expected to draw custom graphical objects or create an elaborate visual design. Instead, you will use standard Qt widgets and layouts to organize controls and information with sensible sizing, alignment, spacing, visual hierarchy, and state feedback. These are useful front-end skills for larger team projects, including future senior design projects.

Unlike a fixed sequence of function calls, a GUI must respond to actions in whatever valid order the user performs them. As you build M3, pay attention not only to what each button does, but also to how one event changes the rest of the interface—especially when processing becomes available, when old output becomes stale, and what should be cleared or preserved.

1. Application Structure and Workflow

The application is organized around one complete workflow. The Workspace and Selected Document areas manage and display loaded documents. Build Corpus prepares the current Workspace for processing. Query/Search presents ranked results through Qt Model/View. Selecting a result exposes its details and source document. Term Inspector queries corpus statistics, and Context builds context from the current query and processing settings.

```
Add documents → select/view a document → Build Corpus → Search
             → select/view a result → View Source / Inspect Term → Build Context
```

These areas are connected. A search is valid only for a corpus built from the current Workspace. If the user adds another document, the Workspace changes and the old corpus is no longer current. The GUI must reflect that change before more processing is allowed.

Application window and presentation

- The application must open at an initial size of 900 × 900 pixels.
- The window must remain resizable. Do not use setFixedSize().
- Use Qt layouts so the interface adjusts appropriately when the window is resized. Do not hard-code individual widget positions.
- Long document/result/context text should remain usable without forcing the entire window to grow.
- Use the same major organization shown in Figure 1: top Workspace/Selected Document pair, full-width Query area, bottom Term Inspector/Context pair, and the status/reset/exit row.

Small platform-dependent differences in spacing, font rendering, and native widget appearance are expected. The requirement is a logically organized, readable, professional interface—not pixel-perfect reproduction.

Color requirement

You must use a non-white background color in each of the following three areas: the application title/header area, the Reset button, and the Exit button. You may choose the colors. The three areas should be visually clear, and the text must remain easy to read. You are not required to use the same colors shown in Figure 1. The rest of the interface should remain visually restrained so that the functional areas are easy to read.

2. Provided Code and M3 Boundaries

M3 is cumulative. The starter provides the M1/M2 headers needed by the project, but it does not provide your completed M2 implementation files. Add your working M2 implementation to the M3 project. Do not reimplement document storage, chunking, retrieval, or context construction inside the GUI.

The required M3 headers are provided and define the interfaces used for grading. Do not modify those required headers or their API contracts. You may add separate helper files/classes or non-member helpers in .cpp files when useful, but the required interfaces must remain unchanged.

Important backend APIs

```
Workspace:
    void add_document(Document document);
    const std::vector<Document>& documents() const noexcept;
    void clear() noexcept;

ProcessingCore:
    void rebuild(const Workspace& workspace);
```

```
    std::size_t chunk_count() const noexcept;
    std::size_t document_frequency(const std::string& term) const;
    std::size_t term_frequency(const std::string& term,
                               const std::string& chunk_id) const;
    std::vector<SearchResult> search(const std::string& query, int k) const;
    std::vector<ContextItem> build_context(const std::string& query,
                                           int k,
                                           std::size_t token_budget) const;
```

Notice that build_context() receives the current query, k, and token budget. It performs the needed search internally; it does not consume the rows currently displayed in the search-results table.

M3 GUI components

Component	Responsibility
MainWindow	Owns Workspace and ProcessingCore; coordinates backend operations and application-wide state.
WorkspaceWidget	Document list, Add Document, Build Corpus, corpus status, and selected-document display.
SearchWidget	Query/k controls, Search request, result table, selected-result details, and View Source.
SearchResultModel	Adapts std::vector<SearchResult> to QTableView.
TermInspectorWidget	Term input and DF/selected-chunk TF display.
ContextWidget	Token-budget slider, Build Context request, and context display.
Resettable	Abstract interface used to reset major GUI components polymorphically.

3. Events, Signals/Slots, Callback, and Polymorphism

Qt signals and slots are the primary event mechanism in M3. A component handles events local to itself and emits a meaningful signal when another part of the application must respond. MainWindow coordinates relationships that involve multiple components or the backend.

You are expected to write the connect() calls and the slot/event-handler implementations. Do not use lambdas; use the signal/slot structure provided by the required headers.

```
Typical event path:
Qt widget signal
    → component slot
    → student-defined component signal
    → MainWindow slot
    → Workspace / ProcessingCore
    → ordinary component update functions
```

Exit callback

M3 deliberately includes one traditional function-pointer callback in addition to Qt signals/slots. MainWindow stores an ExitCallback:

```
using ExitCallback = void (*)(QWidget* parent);

void set_exit_callback(ExitCallback callback) noexcept;
```

When Exit is clicked, the Qt event reaches MainWindow, which invokes the registered callback. The callback must show a QMessageBox asking the user to confirm exit. If the user chooses Yes, quit the application. If the user chooses No, return to the application without changing its current state.

Resettable abstract interface

```
class Resettable {
public:
    virtual ~Resettable() = default;
    virtual void reset() = 0;
};
```

WorkspaceWidget, SearchWidget, TermInspectorWidget, and ContextWidget implement Resettable. MainWindow must use the common Resettable interface when resetting these components so that the reset operation demonstrates virtual dispatch across different GUI component types.

4. Startup State

Area / Control	Required startup state
Workspace	Empty; Add Document enabled; Build Corpus disabled.
Selected Document	Empty and read-only.
Corpus	Not built.
Query	Empty.
Results (k)	QSpinBox, range 1–100, value 3.
Search	Disabled.
Results table	Empty and read-only.
Selected Result / View Source	Empty; View Source disabled.
Term Inspector	Term empty; DF and selected-chunk TF neutral/blank; Inspect disabled.
Token budget	QSlider with visible numeric value; startup value 300.
Build Context	Disabled; context display empty/read-only.
Status	Ready.
Reset / Exit	Enabled.

5. Workspace and Selected Document

Add Document

- Clicking Add Document opens a QFileDialog for selecting a text file.
- If the user cancels the dialog, make no changes.
- For a selected file, read its full text, create a Document, and add it to Workspace.
- Use the absolute file path as the document ID and the file name as the document title.
- Add the title to the Workspace list.
- Because the Workspace changed, invalidate any current corpus and clear processing output that depends on the old corpus.

Adding a document is therefore more than a list update. It changes the data on which later processing depends.

Selecting a document

When a document is selected in the Workspace list, show its title and complete text in Selected Document. The text is read-only. This is a presentation event only: selecting a document must not build the corpus, run Search, inspect a term, or build Context.

6. Build Corpus and Corpus Validity

- Build Corpus is enabled whenever at least one document is in the Workspace.
- Clicking it calls ProcessingCore::rebuild(workspace).
- After a successful rebuild, mark the corpus current for the present Workspace.
- Display processing_core.chunk_count(), for example, “Corpus: 12 chunks”.
- Clear old search results, selected-result details, selected chunk/term statistics, and context after rebuilding.
- Adding another document later makes the corpus stale again. Search, Inspect, and Build Context must not use that stale corpus.

Building the corpus is always an explicit user request. Adding a document does not automatically rebuild it.

7. Query, Search, and SearchResultModel

Control	Required behavior
Query	QLineEdit. Editing it does not automatically run Search.
Results (k)	QSpinBox, range 1–100, startup/reset value 3.

Search	Enabled only when the corpus is current and the effective query contains non-whitespace text.
Results	QTableView backed by SearchResultModel.
Selected Result	Read-only details for the selected SearchResult.
View Source	Enabled only when a valid result row is selected.

On Search, call ProcessingCore::search(query, k), replace the model contents with the returned results, clear any previous table selection, clear selected-result details and context, and update result-dependent state. Zero returned results is a valid successful search.

Search-result table columns

Column	Displayed value
Rank	Result position: 1, 2, 3, ...
Document	SearchResult::document_id
Chunk	SearchResult::chunk_sequence
Score	SearchResult::score
Text	SearchResult::text

A note about QVariant

You will see QVariant in the provided SearchResultModel interface because Qt requires QAbstractTableModel::data() and headerData() to return this type. A QVariant is simply a Qt container that can hold different kinds of values, such as an integer, QString, or double. This allows one model function to return different types of display data for different cells. You do not need to study QVariant as a separate M3 topic.

```
Examples of values that Qt can return as QVariant:
    return 3;                       // integer
    return QString("document.txt"); // text
    return 0.82;                    // floating-point value
```

Implement the required row/column counts, display/header roles, model reset notifications, set_results(), clear(), and result_at(). Retrieval logic does not belong in SearchResultModel.

Result selection and View Source

Selecting a result row displays that SearchResult's metadata and full text without rerunning Search. It also makes the selected chunk available to Term Inspector and enables View Source. Clicking View Source identifies the selected SearchResult's document_id and asks MainWindow to select/display that document in WorkspaceWidget. View Source must not perform a new search.

8. Term Inspector

Term Inspector exposes two existing ProcessingCore statistics in a small GUI tool. It is intentionally connected to both corpus state and result selection.

- Inspect is enabled when the corpus is current. If the term field is empty or contains only whitespace, clicking Inspect makes no inspection request.
- Document frequency uses ProcessingCore::document_frequency(term) and does not require a selected search result.
- If a search-result chunk is selected, also display ProcessingCore::term_frequency(term, selected_chunk_id).
- If no result is selected, document frequency is still valid; selected-chunk TF should remain neutral/blank.
- Selecting a different result changes which chunk would be used for TF, but does not automatically inspect the term.
- When the Workspace changes or the corpus is invalidated/rebuilt, clear stale term statistics.

9. Context

Context uses the current query, current k, and the token budget selected with the slider. It does not depend on which search-result row is selected.

- Use a horizontal QSlider with range 50–1000, step size 50, and startup/reset value 300. Moving the slider updates the visible numeric budget but does not automatically build context.
- Build Context is enabled only when the corpus is current and the effective query is nonempty.
- On click, call ProcessingCore::build_context(current_query, current_k, current_token_budget).
- Display the returned ContextItem values clearly. An empty vector is valid.
- Preserve the backend's truncated flag; do not invent new truncation behavior in the GUI.

```
For each ContextItem, display:
Document: ...
Chunk: ...
Tokens: ...
Score: ...
Truncated: ...
Text: ...
```

10. Keeping Application State Consistent

The main M3 challenge is not creating individual widgets; it is keeping related widgets and backend state consistent as events occur. The central invariant is:

Search, Term Inspector, and Context may use only a corpus that is current for the present Workspace.

Changing an input is not the same event as requesting an operation. Editing the query, changing k, moving the token-budget slider, selecting a document, or selecting a result should update only the appropriate GUI state. Backend processing occurs when the corresponding action is explicitly requested.

Event	Important effects
Add Document	Workspace changes; corpus becomes stale; clear stale results/selection/term statistics/context; processing waits for rebuild.
Build Corpus	Corpus becomes current; show chunk count; clear old processing output.
Edit query	Update action availability only; do not auto-search or auto-clear existing results.
Search	Replace results; clear old selection/selected-result/context; update selected-chunk state.
Select result	Display result; update selected chunk; enable View Source; do not rerun Search.
Change k / budget	Change input only; do not automatically process.
Reset	Restore the complete startup state.

11. Reset and Exit

Reset

Reset is always enabled and returns the application to the same observable state as startup. Clear Workspace, document selection/display, query, results, selected result, selected chunk, term statistics, and context; mark the corpus not current; restore k to 3 and the token budget to 300; restore enabled/disabled states; and show “Application reset.” as the current status.

Use the Resettable interface polymorphically for the major resettable GUI components rather than calling four unrelated reset implementations as separate special cases.

Exit

Exit is always enabled. It must use the registered ExitCallback described earlier. The callback asks for confirmation with QMessageBox. Yes quits; No returns to the application unchanged.

12. Example Complete Session

- Add two text files. Both titles appear in Workspace.
- Select one document and confirm that its title and complete text appear in Selected Document.
- Click Build Corpus and confirm that the corpus status reports the chunk count.
- Enter a query such as “signals and slots”, leave Results at 3 (or choose another k), and click Search.
- Confirm the table shows Rank, Document, Chunk, Score, and Text. Select a row and inspect its details.

- Click View Source and confirm that the source document becomes selected/displayed without rerunning Search.
- Enter a term and click Inspect. Confirm DF appears; with a selected result, selected-chunk TF also appears.
- Choose a token budget and click Build Context. Confirm the returned ContextItem information is displayed.
- Add a third document. Confirm the old corpus becomes stale, stale processing output is cleared, and processing requires Build Corpus again.
- Click Reset and confirm the startup state is restored. Click Exit and test both No and Yes in the confirmation dialog.

13. Keep M3 Focused

M3 is a focused Qt application for selected AI Workspace functionality. Do not add document removal/editing, strategy configuration, chunk browsing, sorting/filtering, tabs, menus/toolbars, drag-and-drop, multiple windows, IPC, threads, background processing, or other unrelated features. Visual creativity is not a grading objective.

The required headers already establish the component interfaces. Your work is to implement the .cpp files: construct and organize the widgets, connect events, emit/handle signals, implement the model, integrate the backend, and maintain correct application state. Do not introduce controller/service frameworks or other architectural patterns that are not part of the course.

Before You Submit

- The application builds and opens at 900 × 900, and the window remains resizable.
- Qt layouts keep the interface usable when the window is resized.
- Header, Reset, and Exit each have a readable non-white background color of your choice.
- Startup/reset values are correct: Corpus = Not built, Results = 3, Token budget = 300.
- Search, Inspect, Build Context, View Source, and Build Corpus are enabled only when appropriate.
- Adding a document after processing correctly invalidates the corpus and clears stale processing output.
- Search results are displayed through SearchResultModel/QTableView.
- Result selection does not rerun Search; input changes do not automatically run processing.
- View Source selects/displays the result's source document.
- Term Inspector supports DF without a selected result and TF when a result chunk is selected.
- Reset uses the Resettable abstraction; Exit uses the function-pointer callback and confirmation dialog.
- You have walked through the complete session above.